Report - Version 1: Baseline Agent

Introduction

Version 1 is the first working version of the Building Agentic AutoML project.

The goal is to build a minimal AutoML agent that receives a dataset and target, detects the task, trains a baseline, and returns a structured state.

This version focuses on the fundamental flow:

```
Dataset + target
↓
Detect task and features
↓
Prepare data and baseline
↓
Evaluate
↓
Return the state
```

The architecture is intentionally simple: one model, one train-validation split, and no optimization. The purpose is to understand the core behavior before introducing more advanced components.

Goal of Version 1

The goal of Version 1 is to create a minimal but functional agent.

The agent should be able to:

- load a dataset and target;
- distinguish classification from regression;
- identify numerical and categorical features;
- select a suitable model and metric;
- train, evaluate, and return the state.

This is not a complete AutoML system yet: it is the foundation of the project.

Architecture

The flow of Version 1 is:

```
Dataset + Target → Task/Features → Preparation → Model/Metric → Evaluation → State
```

In this version, the agent function directly coordinates all operations.

Task detection, preparation, model choice, evaluation, and state construction are therefore still concentrated in one flow.

Task Detection

The first decision step is to inspect the target column.

The initial rule is intentionally simple: if the target contains at most 20 unique values, the problem is considered classification; otherwise it is considered regression.

```
def detect_task(df, target):
    n_unique = df[target].nunique()
    if n_unique <= 20:
        return "classification"
    return "regression"
```

On the binary dataset used in the notebook, the result is:

```
"classification"
```

Feature Detection

The agent separates numerical and categorical features by reading the dataframe dtypes directly.

For the classification dataset, it detects:

```
numerical = ['num_1', 'num_2', 'num_3', 'num_4']
categorical = ['category_1', 'category_2']
```

This avoids manual feature configuration in the first version and allows the flow to adapt automatically to the tabular structure.

Data Preparation

LightGBM can handle categorical variables directly. The agent therefore converts categorical columns to the pandas category dtype and separates the feature matrix X from the target y.

```
for column in categorical_features:
    data[column] = data[column].astype("category")
X = data.drop(columns=target)
y = data[target]
```

Baseline Model

The model is selected according to the detected task.

```
classification → LGBMClassifier
regression     → LGBMRegressor
```

Both use random_state=42 and the default LightGBM configuration. No hyperparameter search is performed yet.

Metric Selection

The metric is also selected automatically according to the task:

```
classification → ROC AUC
regression     → RMSE
```

The same agent can therefore use a suitable evaluation measure without requiring additional manual configuration.

Training and Evaluation

The agent splits the dataset into training and validation sets with test_size=0.2 and random_state=42.

For classification, it also uses stratify=y so that class proportions are preserved across the two subsets.

After fitting the model, evaluation depends on the task:

```
classification: predict_proba → roc_auc_score
regression:     predict       → RMSE
```

On the binary dataset included in the project, the notebook obtains:

```
model  = LGBMClassifier
metric = roc_auc
score  = 0.9068167604752971
```

The result confirms that the baseline is already operational on the example dataset.

Agent

The agent function combines all previous components into a single flow.

In Version 1, the Agent performs the following operations:

1. loads the dataset and target;

2. detects classification or regression;

3. identifies and prepares the features;

4. selects the model and metric;

5. trains and evaluates the baseline;

6. returns the final state.

A typical state looks like this:

```
{
  "task": "classification",
  "model": "LGBMClassifier",
  "metric": "roc_auc",
  "score": 0.9068167604752971
}
```

The state makes the main decisions and evaluation result explicit.

Regression Test

The same agent is also tested on a second dataset by changing only the input file.

The agent detects the new task automatically and produces:

```
task   = regression
model  = LGBMRegressor
metric = rmse
score  = 1.8760451113860066
```

The test shows the basic AutoML behavior: model and metric change without changing the agent logic.

What Version 1 Teaches

Version 1 teaches the fundamental logic of an AutoML agent.

The main idea is:

```
observe → decide → prepare → train → evaluate → return
```

The notebook shows how a machine learning process can be transformed into a sequence of automatic decisions coordinated by a single agent.

It also introduces the state as a compact representation of the decisions and results produced by the system.

Limitations

Version 1 has several natural limitations:

- task detection relies only on the number of unique target values;
- only one model family, LightGBM, is used;
- there is no comparison across multiple algorithms;
- there is no hyperparameter search;
- there are no preprocessing strategies beyond categorical conversion;
- evaluation uses a single train-validation split;
- there is no cross-validation;
- there is no advanced handling of missing values, outliers, or data leakage;
- there is no planner or multi-agent architecture yet.

These limitations are intentional.

They define the roadmap for the next versions.

Conclusion

Version 1 is the foundation of the Building Agentic AutoML project.

It creates the first working agent that can move from a tabular dataset to an evaluated baseline model while automatically adapting between classification and regression.

The most important lesson is that agentic AutoML is not only about training a model.

It is a system that observes data, makes decisions about the task and features, selects suitable modeling tools, evaluates the result, and returns an interpretable state.

The natural next step is to expand the agent's decision-making capabilities with additional components and strategies so the AutoML process becomes progressively more autonomous and robust.